AI Lawyer India: Solution Doc (Problem 2)

[Name] · [Roll no.] · [Branch] | Prototype: **https://ai-lawyer-eta.vercel.app** | Raw outputs: `outputs/raw/` (17 cases) | Date: 8 Nov 2026

1. Problem choice and scope

I picked Problem 2 because everyday legal problems in India (money not returned, a scam, a notice nobody explains) hurt many people and need no show-specific context. I went deep on **money disputes** and said what is not covered. **Covered, with verified law:** online refunds and non-delivery, e-commerce platforms, payment scams and unauthorised transactions, rental deposits, unpaid salary, private loans where documents are held, and demand notices and cheque bounce. **Not covered:** family, property title, tax, criminal defence. For those the tool still gives practical steps but labels them "(General)" and does not cite law. It never stops at "see a lawyer".

2. Assumptions (the brief was silent)

- Users are ordinary people on a phone, often not writing English; text or voice, replies in their language (script detected in code).
- "Today" is the server date; criminal law means the **BNS 2023**, not the IPC. Consumer forum limits use the amount *paid* (District up to Rs 50 lakh).
- indiacode.nic.in blocked my fetches, so **33 of 53 corpus entries come from secondary sources** (bare-act sites, law-firm notes); 20 are official text. Each entry carries `source_type` and `verified_on`.
- Free tiers only: Gemini free API, Vercel Hobby. No user data is stored; the case lives in the browser and is sent with each request.

3. How it works: what AI does, what code does

Flow: (1) tell your story, (2) it restates a structured case file for you to confirm, (3) it explains where you stand, cites laws and asks what proof you have, (4) it gives a step-by-step plan, the right forum, a draft notice/complaint/reply, and a verdict on whether this tool can handle the case, (5) follow-up questions, plus **Practice mode**: the AI plays the other side's lawyer, briefs you on likely tricks, asks hard questions and scores each answer.

Gemini (3.5 Flash-Lite)	Turns the messy story into facts and a timeline; explains in plain language; picks relevant provisions *from the file*; drafts documents; plays the opposing lawyer.
Plain code	Stage order; the law file; **citation validation**; flagging of unknown section numbers, "IPC" and case names; age of the claim from the event date; language detection and a retry if an English user gets Hindi; **verdict rules** (amount over Rs 50 lakh, claim near its time limit, threats, a court case already filed, no matching law).
Human in the loop	The corpus is hand-reviewed with sources. The verdict recommends a lawyer, *in addition to* steps, for the risky cases above. Drafts have `[PLACEHOLDERS]` and a "check every detail" note.

Swappable model: one `llm()` function; set `LLM_PROVIDER=openai` with a key, model and base URL to use any OpenAI-compatible API, or change `GEMINI_MODEL`.

4. How it avoids invented laws

- **Closed corpus.** 53 entries (about 7k tokens) go in every prompt. The model may cite only by entry id; code drops any id not in the file and tells the user.
- **Safety net on free text.** Section/rule/article numbers that appear nowhere in the file, "IPC", and "X v. Y" case citations are flagged to the user as unverified.
- **Honest caveats.** Each entry says what is state-specific, a model law or unconfirmed (e.g. the Model Tenancy Act applies only where a state adopted it; RBI liability rules change for transactions from 1 Jan 2027; 3 entries are marked *unverified*). Prompts forbid fixed refund deadlines, case law and certainty.
- **Limit:** the model can still paraphrase an entry loosely. The citation check proves the entry exists, not that the summary fits.

5. How I checked quality (real runs, honest failures)

I wrote **17 test conversations** and ran them on the live site. Checks: verdict, issue type, a cited law id, reply language, forbidden text. Cases: refunds in English, Hindi and Telugu; three scam types; a Rs 60 lakh claim; a 30-month-old claim; salary; deposit; Telugu private-loan documents; cheque notice; blackmail; a trap asking for "IPC 420 and a Supreme Court case"; a vague one-line message. Final: 17/17 pass, but the path there is the useful part:

Run	What failed and the fix
13 cases, run 1: 12/13	The model said "7 months" for a May 2024 event (it does not know today's date), so a time-barred claim was called "can help". **Fix:** model extracts the date, code computes the age.
After adding salary, deposit, loan, cheque: 14/17	(a) The model set "court case exists" for a plain company notice, giving the wrong "lawyer" verdict. **Fix:** renamed the field to `court_case_filed` and tightened the prompt. (b) Gemini 503/429 overloads and a Vercel timeout: added retries. (c) Two failures were my test script giving up too early.
Earlier, by hand	English messages got Hindi or Hinglish replies (the prompt listed Hindi as an example). **Fix:** language detected in code, enforced, one retry if wrong.

Where it still fails or is weak: (1) my cases test behaviour, not legal correctness; no lawyer has reviewed the output. (2) The cheque-bounce case gets "can help" although a bounced cheque carries criminal exposure; the verdict is too optimistic there. (3) Practice mode sometimes states a detail slightly wrong (said "a whole month" for about three weeks). (4) Deposit and wage law is state-specific, so answers are often general guidance. (5) Replies take 5-25 s on the free tier, and a stalled Gemini call can exceed Vercel's 60 s limit (one run failed this way). (6) Voice input depends on the browser; voice output uses the browser's voice.

6. AI tools used, and where they got it wrong

Claude Code wrote the code and helped draft the corpus; three Claude research sub-agents checked each legal claim against web sources (they were told to write NOT VERIFIED, not guess); **Gemini 3.5 Flash-Lite** runs inside the app. **Where they were wrong:** (1) Claude's recalled section numbers were wrong twice and the source check caught both: Sale of Goods Act *s.56* is damages for *non-acceptance* (non-delivery is *s.57*), and Consumer Protection Act *s.51* is an appeal, not State jurisdiction. This is why the corpus is built from sources, not memory. (2) Gemini mis-computed the age of a claim (above). (3) The default model `gemini-2.5-flash` had been retired for new users; larger Gemini models took 15-60 s per call, so I chose a lite one.

7. Scale and cost

A case takes about **5-7 model calls**: intake 1-3 (small prompt), confirm + analysis, plan, then follow-ups. The corpus (about 7k tokens) is sent with analysis, plan, follow-up and practice calls, so roughly **25k input and 3k output tokens per case**. I have not measured rupee cost; it is those tokens times the chosen model's price. The server is stateless, so it scales horizontally and stores no personal data. **At volume:** a paid model (one env change); cache the fixed corpus prefix; once the corpus passes a few hundred entries, send only entries matching the detected issue type; queue and rate-limit calls; log anonymised failures for lawyer review; re-verify entries on a schedule (each has `verified_on`) because law moves (RBI rules 2027, new labour codes, state rent laws).

8. Next steps

Have a lawyer review the corpus and 20 outputs; verify the 3 unverified and 33 secondary entries against official text; add a cheque-bounce "criminal exposure" rule; add state-specific rent and money-lender entries; measure real cost and latency; test with real users in Hindi, Telugu and Tamil.